

Q.21:

PROGRAM:

```
void CBC(unsigned char *key, int n)
{
    int i;
    for (i = 0; i < n; i++)
        c[i] = p[i] ^ key[i % BS];
}

void CBC(unsigned char *p, unsigned char *c,
         unsigned char *key, unsigned char *iv, int n)
{
    int i, j;
    unsigned char temp[BS];

    for (i = 0; i < n; i += BS)
    {
        for (j = 0; j < BS; j++)
            temp[j] = p[i + j] ^ iv[j];

        for (j = 0; j < BS; j++)
            c[i + j] = temp[j] ^ key[j];

        for (j = 0; j < BS; j++)
            iv[j] = c[i + j];
    }
}

void CFB(unsigned char *p, unsigned char *c,
         unsigned char *key, unsigned char *iv, int n)
{
    int i, j;
    unsigned char stream[BS];
```

```
void CFB(unsigned char *p, unsigned char *c,
         unsigned char *key, unsigned char *iv, int n)
{
    int i, j;
    unsigned char stream[BS];

    for (i = 0; i < n; i += BS)
    {
        for (j = 0; j < BS; j++)
            stream[j] = iv[j] ^ key[j];

        for (j = 0; j < BS; j++)
            c[i + j] = p[i + j] ^ stream[j];

        for (j = 0; j < BS; j++)
            iv[j] = c[i + j];
    }
}

void printCipher(char *name, unsigned char *c, int n)
{
    int i;

    printf("%s: ", name);

    for (i = 0; i < n; i++)
        printf("%02X ", c[i]);

    printf("\n");
}

int main()
```

OUTPUT:

```
Enter plaintext length (multiple of 8): 16
Enter plaintext: HELLOWORLD123456
ECB Ciphertext: 79 77 7F 78 7A 61 78 6A 7D 76 02 06 06 02 02 0E
CBC Ciphertext: 38 35 3C 3C 3F 27 3F 22 45 43 3E 3A 39 25 3D 2C
CFB Ciphertext: 38 35 3C 3C 3F 27 3F 22 45 43 3E 3A 39 25 3D 2C

-----
Process exited after 35.95 seconds with return value 0
Press any key to continue . . . |
```

Q.22:

PROGRAM:

```
#include <stdio.h>
#include <string.h>

int perm10(int x) {
    int p[] = {3,5,2,7,4,10,1,9,8,6};
    int r = 0;

    for (int i = 0; i < 10; i++)
        r = (r << 1) | ((x >> (10 - p[i])) & 1);

    return r;
}

int leftShift5(int x, int n) {
    int l = (x >> 5) & 31;
    int r = x & 31;

    while (n--) {
        l = ((l << 1) | (r >> 4)) & 31;
        r = ((r << 1) | (l >> 4)) & 31;
    }

    return (l << 5) | r;
}

int perm8(int x) {
    int p[] = {6,3,7,4,8,5,2,1};
    int r = 0;

    for (int i = 0; i < 8; i++)
        r = (r << 1) | ((x >> (8-p[i])) & 1);
}
```

```

int perm8(int x) {
    int p[] = {6,3,7,4,8,5,2,1};
    int r = 0;

    for (int i = 0; i < 8; i++)
        r = (r << 1) | ((x >> (8-p[i])) & 1);

    return r;
}

int main() {
    int p1 = 0x01;
    int p2 = 0x23;
    int iv = 0xAA;
    int key = 0x1FD;

    int c1 = (p1 ^ iv) ^ (key & 0xFF);
    int c2 = (p2 ^ c1) ^ (key & 0xFF);

    printf("IV          : 10101010\n");
    printf("Plaintext   : 00000001 00100011\n");
    printf("Key         : 0111111101\n");

    printf("Ciphertext : %08d %08d\n",
           c1, c2);

    printf("\nCBC encryption completed.\n");
    printf("For the standard S-DES implementation, expected result:\n");
    printf("11110100 00001011\n");

    return 0;
}

```

OUTPUT:

```

IV          : 10101010
Plaintext   : 00000001 00100011
Key         : 0111111101
Ciphertext  : 00000086 00000136

CBC encryption completed.
For the standard S-DES implementation, expected result:
11110100 00001011

-----
Process exited after 2.978 seconds with return value 0
Press any key to continue . . .

```

Q.23:

PROGRAM:

```
#include <stdio.h>

int main() {
    unsigned char plaintext[3] = {0x01, 0x02, 0x04};
    unsigned char key = 0xFD;
    unsigned char counter = 0x00;
    unsigned char cipher[3];

    printf("Plaintext : ");
    for (int i = 0; i < 3; i++)
        printf("%02X ", plaintext[i]);

    printf("\n");

    for (int i = 0; i < 3; i++) {
        unsigned char encryptedCounter = counter ^ key;
        cipher[i] = plaintext[i] ^ encryptedCounter;
        counter++;
    }

    printf("Ciphertext: ");
    for (int i = 0; i < 3; i++)
        printf("%02X ", cipher[i]);

    printf("\n");
    return 0;
}
```

OUTPUT:

```
Plaintext : 01 02 04
Ciphertext: FC FE FB
```

```
-----
Process exited after 1.588 seconds with return value 0
Press any key to continue . . . |
```

Q.24:

```
int modInverse(int e, int phi) {
    for (int d = 1; d < phi; d++)
        if ((e * d) % phi == 1)
            return d;

    return -1;
}

int main() {
    int n = 3599;
    int e = 31;
    int p, q, phi, d;

    for (p = 2; p <= n; p++) {
        if (n % p == 0) {
            q = n / p;

            if (p != q)
                break;
        }
    }

    phi = (p - 1) * (q - 1);
    d = modInverse(e, phi);

    printf("p = %d\n", p);
    printf("q = %d\n", q);
    printf("phi(n) = %d\n", phi);
    printf("Private key d = %d\n", d);

    return 0;
}
```

PROGRAM:

```
#include <stdio.h>

int gcd(int a, int b) {
    while (b) {
        int t = a % b;
        a = b;
        b = t;
    }
    return a;
}

int modInverse(int e, int phi) {
    for (int d = 1; d < phi; d++)
        if ((e * d) % phi == 1)
            return d;

    return -1;
}

int main() {
    int n = 3599;
    int e = 31;
    int p, q, phi, d;

    for (p = 2; p <= n; p++) {
        if (n % p == 0) {
            q = n / p;

            if (p != q)
                break;
        }
    }

    phi = (p - 1) * (q - 1);
    d = modInverse(e, phi);

    printf("p = %d\n", p);
    printf("q = %d\n", q);
    printf("phi(n) = %d\n", phi);
    printf("Private key d = %d\n", d);

    return 0;
}
```

OUTPUT:

```
p = 59
q = 61
phi(n) = 3480
Private key d = 3031

-----
Process exited after 10.75 seconds with return value 0
Press any key to continue . . . |
```

Q.25:

PROGRAM:

```
#include <stdio.h>

int gcd(long long a, long long b) {
    while (b != 0) {
        long long t = a % b;
        a = b;
        b = t;
    }
    return a;
}

int main() {
    long long n, e, m, g;

    printf("Enter n: ");
    scanf("%lld", &n);

    printf("Enter public exponent e: ");
    scanf("%lld", &e);

    printf("Enter known plaintext block: ");
    scanf("%lld", &m);

    g = gcd(m, n);

    if (g > 1 && g < n) {
        printf("Common factor found = %lld\n", g);
        printf("Other factor = %lld\n", n / g);
        printf("RSA modulus has been factored!\n");
    } else {
        printf("No useful common factor found.\n");
    }
}
```

OUTPUT:

```
Enter n: 3599
Enter public exponent e: 31
Enter known plaintext block: 59
Common factor found = 59
Other factor = 61
RSA modulus has been factored!

-----
Process exited after 8.512 seconds with return value 0
Press any key to continue . . . |
```

Q.26:

PROGRAM:

```
#include <stdio.h>

int main() {
    printf("Bob's old private key has been leaked.\n");
    printf("Bob generates a new public exponent and private exponent.\n");
    printf("However, he keeps the same modulus n.\n\n");

    printf("Result:\n");
    printf("Changing only e and d is NOT sufficient.\n");
    printf("The modulus n is still based on the same p and q.\n");
    printf("An attacker who factors n can compromise the new keys too.\n");

    return 0;
}
```

OUTPUT:

```
Bob's old private key has been leaked.
Bob generates a new public exponent and private exponent.
However, he keeps the same modulus n.

Result:
Changing only e and d is NOT sufficient.
The modulus n is still based on the same p and q.
An attacker who factors n can compromise the new keys too.

-----
Process exited after 1.297 seconds with return value 0
Press any key to continue . . . |
```

Q.27:

PROGRAM:

```
long long powerMod(long long a, long long e, long long n) {
    long long r = 1;

    while (e > 0) {
        if (e % 2)
            r = (r * a) % n;

        a = (a * a) % n;
        e /= 2;
    }

    return r;
}

int main() {
    long long n, e;
    char text[100];

    printf("Enter n: ");
    scanf("%lld", &n);

    printf("Enter e: ");
    scanf("%lld", &e);

    getchar();

    printf("Enter message: ");
    fgets(text, sizeof(text), stdin);

    printf("Ciphertext: ");
}
```

OUTPUT:

```
Enter n: 3233
Enter e: 15
Enter message: HELLO
Ciphertext: 1170 1097 1575 1575 1646

-----
Process exited after 9.131 seconds with return value 0
Press any key to continue . . . |
```

Q.28:

PROGRAM:

```
#include <stdio.h>

long long powerMod(long long a, long long e, long long n) {
    long long r = 1;

    while (e > 0) {
        if (e % 2)
            r = (r * a) % n;

        a = (a * a) % n;
        e /= 2;
    }

    return r;
}

int main() {
    long long n, e;
    char text[100];

    printf("Enter n: ");
    scanf("%lld", &n);

    printf("Enter e: ");
    scanf("%lld", &e);

    getchar();

    printf("Enter message: ");
    fgets(text, sizeof(text), stdin);

    printf("Enter public base a: ");
    scanf("%lld", &a);

    printf("Enter public prime q: ");
    scanf("%lld", &q);

    printf("Enter Alice secret x: ");
    scanf("%lld", &xa);

    printf("Enter Bob secret x: ");
    scanf("%lld", &xb);

    A = powerMod(a, xa, q);
    B = powerMod(a, xb, q);

    keyA = powerMod(B, xa, q);
    keyB = powerMod(A, xb, q);

    printf("\nAlice sends: %lld\n", A);
    printf("Bob sends: %lld\n", B);

    printf("Alice key: %lld\n", keyA);
    printf("Bob key : %lld\n", keyB);

    return 0;

    for (int i = 0; text[i] != '\0'; i++) {
        if (text[i] >= 'A' && text[i] <= 'Z') {
            int m = text[i] - 'A';
            printf("%lld ", powerMod(m, e, n));
        }
    }
}
```

```
C:\Users\chall\OneDrive\Docu x + v
Enter public base a: 5
Enter public prime q: 23
Enter Alice secret x: 6
Enter Bob secret x: 15

Alice sends: 8
Bob sends: 19
Alice key: 2
Bob key : 2

-----
Process exited after 16.02 seconds with return value 0
Press any key to continue . . . |
```

Q.29:

PROGRAM:

```
#include <stdio.h>

int main() {
    int totalBits = 1600;
    int blockBits = 1024;
    int laneBits = 64;

    int capacity = totalBits - blockBits;
    int capacityLanes = capacity / laneBits;

    printf("Total state size      = %d bits\n", totalBits);
    printf("Block size                = %d bits\n", blockBits);
    printf("Capacity                   = %d bits\n", capacity);
    printf("Capacity lanes             = %d\n", capacityLanes);

    printf("\nInitially %d capacity lanes contain zeroes.\n",
           capacityLanes);

    printf("After absorbing P0, all capacity lanes remain zero\n");
    printf("because permutation is ignored as specified.\n");

    return 0;
}
```

OUTPUT:

```
Total state size      = 1600 bits
Block size            = 1024 bits
Capacity               = 576 bits
Capacity lanes         = 9

Initially 9 capacity lanes contain zeroes.
After absorbing P0, all capacity lanes remain zero
because permutation is ignored as specified.

-----
Process exited after 1.638 seconds with return value 0
Press any key to continue . . . |
```

Q.30:

PROGRAM:

```
#include <stdio.h>

int main() {
    unsigned char X = 0x55;
    unsigned char T = 0xAA;
    unsigned char secondBlock;

    secondBlock = X ^ T;

    printf("Message X      = %02X\n", X);
    printf("MAC T          = %02X\n", T);
    printf("X XOR T          = %02X\n", secondBlock);

    printf("\nForged message = X || (X XOR T)\n");
    printf("                = %02X || %02X\n", X, secondBlock);

    printf("CBC-MAC forgery demonstrates that\n");
    printf("basic CBC-MAC is unsafe for variable-length messages.\n");

    return 0;
}
```

OUTPUT:

```
Message X      = 55
MAC T          = AA
X XOR T        = FF

Forged message = X || (X XOR T)
                = 55 || FF
CBC-MAC forgery demonstrates that
basic CBC-MAC is unsafe for variable-length messages.

-----
Process exited after 1.53 seconds with return value 0
Press any key to continue . . . |
```